

Volume 06 - Core Specifications

Version v25 draft

spec-06-core-01-kernel-specification.md

Kernel Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-001

Related Blueprint Documents

- MAL-V03-B03-01-002
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for kernel boot, trusted service access, invariant enforcement, and orderly shutdown in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for kernel boot, trusted service access, invariant enforcement, and orderly shutdown.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.

- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.
- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/kernel/status
- POST /core/kernel/commands
- GET /core/kernel/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- KernelCommandReceived
- KernelCommandAccepted
- KernelCommandRejected
- KernelStateChanged
- KernelFailureDetected
- KernelRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.
- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-001
- MAL-V03-B03-01-002
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Lifecycle Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-002

Related Blueprint Documents

- MAL-V03-B03-01-003
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for core lifecycle states, transition validation, readiness gates, degradation, maintenance, and shutdown in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for core lifecycle states, transition validation, readiness gates, degradation, maintenance, and shutdown.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/lifecycle/status
- POST /core/lifecycle/commands
- GET /core/lifecycle/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- LifecycleCommandReceived
- LifecycleCommandAccepted
- LifecycleCommandRejected

- LifecycleStateChanged
- LifecycleFailureDetected
- LifecycleRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-002
- MAL-V03-B03-01-003
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Runtime Manager Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-003

Related Blueprint Documents

- MAL-V03-B03-01-004
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for runtime supervision, start and stop rules, failure handling, restart policy, and runtime health in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime supervision, start and stop rules, failure handling, restart policy, and runtime health.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/runtimemanager/status
- POST /core/runtimemanager/commands
- GET /core/runtimemanager/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- RuntimeManagerCommandReceived
- RuntimeManagerCommandAccepted
- RuntimeManagerCommandRejected

- `RuntimeManagerStateChanged`
- `RuntimeManagerFailureDetected`
- `RuntimeManagerRecovered`

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-003
- MAL-V03-B03-01-004
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Runtime Registry Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-004

Related Blueprint Documents

- MAL-V03-B03-01-005
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for runtime manifest registration, compatibility metadata, deprecation, and lookup behavior in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime manifest registration, compatibility metadata, deprecation, and lookup behavior.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/runtimeregistry/status
- POST /core/runtimeregistry/commands
- GET /core/runtimeregistry/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- RuntimeRegistryCommandReceived
- RuntimeRegistryCommandAccepted
- RuntimeRegistryCommandRejected

- RuntimeRegistryStateChanged
- RuntimeRegistryFailureDetected
- RuntimeRegistryRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-004
- MAL-V03-B03-01-005
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Event Bus Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-005

Related Blueprint Documents

- MAL-V03-B03-01-006
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for core event envelope, publication, subscription, delivery failure, dead- letter, and correlation behavior in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for core event envelope, publication, subscription, delivery failure, dead-letter, and correlation behavior.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/eventbus/status
- POST /core/eventbus/commands
- GET /core/eventbus/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- EventBusCommandReceived
- EventBusCommandAccepted
- EventBusCommandRejected

- EventBusStateChanged
- EventBusFailureDetected
- EventBusRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-005
- MAL-V03-B03-01-006
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Service Container Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-006

Related Blueprint Documents

- MAL-V03-B03-01-007
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for service registration, dependency resolution, lifecycle hooks, privileged services, and disposal in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for service registration, dependency resolution, lifecycle hooks, privileged services, and disposal.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/servicecontainer/status
- POST /core/servicecontainer/commands
- GET /core/servicecontainer/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- ServiceContainerCommandReceived
- ServiceContainerCommandAccepted
- ServiceContainerCommandRejected

- ServiceContainerStateChanged
- ServiceContainerFailureDetected
- ServiceContainerRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-006
- MAL-V03-B03-01-007
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Configuration Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-007

Related Blueprint Documents

- MAL-V03-B03-01-008
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for configuration loading, validation, secret separation, environment overrides, and safe inspection in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for configuration loading, validation, secret separation, environment overrides, and safe inspection.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/configuration/status
- POST /core/configuration/commands
- GET /core/configuration/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- ConfigurationCommandReceived
- ConfigurationCommandAccepted
- ConfigurationCommandRejected

- ConfigurationStateChanged
- ConfigurationFailureDetected
- ConfigurationRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-007
- MAL-V03-B03-01-008
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Identity Access Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-008

Related Blueprint Documents

- MAL-V03-B03-01-009
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for identity context, tenant context, roles, permissions, authorization decisions, and access audit in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for identity context, tenant context, roles, permissions, authorization decisions, and access audit.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/identityaccess/status
- POST /core/identityaccess/commands
- GET /core/identityaccess/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- IdentityAccessCommandReceived
- IdentityAccessCommandAccepted
- IdentityAccessCommandRejected

- IdentityAccessStateChanged
- IdentityAccessFailureDetected
- IdentityAccessRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-008
- MAL-V03-B03-01-009
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

State Manager Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-009

Related Blueprint Documents

- MAL-V03-B03-01-010
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for controlled state transitions, snapshots, journals, recovery checkpoints, and consistency rules in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for controlled state transitions, snapshots, journals, recovery checkpoints, and consistency rules.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/statemanager/status
- POST /core/statemanager/commands
- GET /core/statemanager/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- StateManagerCommandReceived
- StateManagerCommandAccepted
- StateManagerCommandRejected

- StateManagerStateChanged
- StateManagerFailureDetected
- StateManagerRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-009
- MAL-V03-B03-01-010
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.

Health Manager Specification

Version: v25 draft

Status: Draft

Specification ID: MAL-V06-CORE-SPEC-010

Related Blueprint Documents

- MAL-V03-B03-01-013
- MAL-V03-B03-01-001
- MAL-V03-B03-01-016
- MAL-V05-STD-001
- MAL-V05-STD-013

Purpose

Define executable requirements for readiness, liveness, dependency checks, degradation, incident signals, and health reporting in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for readiness, liveness, dependency checks, degradation, incident signals, and health reporting.

Responsibilities

- Convert the related blueprint into implementation-ready requirements.
- Define what the subsystem must accept, produce, persist, emit, secure, observe, and test.
- Preserve compatibility with Enterprise Core governance, runtime, identity, event, storage, and observability standards.
- Provide acceptance criteria suitable for engineering review before code is written.

Functional Requirements

- The subsystem **MUST** expose a deterministic lifecycle with initialized, ready, degraded, maintenance, stopping, stopped, and failed behavior where applicable.
- The subsystem **MUST** validate caller context before protected reads, writes, transitions, registrations, or privileged operations.
- The subsystem **MUST** emit canonical events for accepted, rejected, failed, recovered, and completed operations.
- The subsystem **MUST** reject invalid inputs with structured errors and auditable rationale.
- The subsystem **MUST** support idempotent retry where repeated requests could otherwise create duplicate side effects.
- The subsystem **MUST** provide queryable status for operators and dependent services.

- The subsystem **MUST** integrate with configuration, identity access, event bus, storage, security, and observability interfaces.

Non-Functional Requirements

- Startup and readiness checks **SHOULD** complete within an operator-defined threshold.
- Protected operations **MUST** be auditable with correlation IDs and tenant context.
- Interfaces **MUST** remain stable across compatible draft releases unless a migration note is recorded.
- Data persistence **MUST** survive restart when the data affects authority, lifecycle, health, identity, or audit.
- Error handling **MUST** avoid leaking secrets, credentials, private payloads, or classified context.
- The subsystem **SHOULD** degrade safely instead of failing open.

Inputs

- Caller identity, tenant context, role bindings, and authorization claims.
- Configuration values and non-secret environment settings.
- Operation commands, manifests, lifecycle requests, event envelopes, or state transition requests.
- Dependency status from Enterprise Core services.

Outputs

- Structured success or failure results.
- Updated subsystem state, registry entries, configuration snapshots, health records, or audit records.
- Canonical events and observability signals.
- Operator-readable diagnostics with sensitive values redacted.

Public Interfaces

- GET /core/healthmanager/status
- POST /core/healthmanager/commands
- GET /core/healthmanager/events
- Administrative inspection interface for authorized operators.

Internal Interfaces

- Kernel service lookup.
- Identity access authorization guard.
- Event bus publication and subscription.
- Core storage read and write adapter.
- Observability metrics, logs, and traces.
- Configuration provider and secret provider where applicable.

Events

- HealthManagerCommandReceived
- HealthManagerCommandAccepted
- HealthManagerCommandRejected

- HealthManagerStateChanged
- HealthManagerFailureDetected
- HealthManagerRecovered

Storage Requirements

- Persist authoritative records that affect lifecycle, access, audit, registry, or recovery.
- Store immutable audit references for protected operations.
- Maintain version history for configuration, manifests, state transitions, or policy-sensitive records.
- Support backup and restore for records required during disaster recovery.

Security Requirements

- Enforce least privilege and tenant isolation for all protected operations.
- Deny by default when caller context, tenant context, role, or policy status is missing.
- Redact secrets and sensitive payloads from logs, metrics, events, and diagnostics.
- Audit privileged, denied, failed, recovered, and policy-exception operations.
- Never allow generated or inferred data to become authoritative without an acceptance path.

Observability Requirements

- Emit metrics for request count, success count, rejection count, error count, latency, degradation, and recovery.
- Attach correlation IDs to events, logs, traces, and audit records.
- Expose readiness and liveness indicators.
- Provide structured diagnostics for operators without exposing sensitive content.

Failure Modes

- Invalid command or malformed payload.
- Unauthorized or ambiguous caller context.
- Dependency unavailable or degraded.
- Storage write failure or stale read.
- Event publication failure.
- Configuration mismatch or missing required setting.

Recovery Rules

- Retry idempotent operations with bounded attempts.
- Move unsafe operations into rejected or degraded state rather than failing open.
- Record recovery attempts and outcomes.
- Rebuild derived indexes or status views from canonical records where possible.
- Require operator review for repeated or high-impact recovery failures.

Test Requirements

- Unit tests for validation, state changes, and error handling.

- Integration tests across identity access, event bus, storage, configuration, and observability.
- Security tests for denied access, tenant isolation, privileged operations, and redaction.
- Recovery tests for dependency failure, restart, retry, and event failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- Functional and non-functional requirements are implementable and testable.
- Public and internal interfaces are explicit.
- Events, storage, security, observability, failure, and recovery behavior are defined.
- Traceability links this specification to blueprint, standards, and future tests.

Implementation Notes

- Start with the smallest implementation slice that satisfies lifecycle, authorization, event, storage, and observability contracts.
- Keep provider, framework, or database choices replaceable until infrastructure specifications lock them down.
- Prefer structured schemas and typed objects over ad hoc dictionaries or free-form strings.
- Treat every TODO in implementation as a traceable issue, not hidden design debt.

Traceability

- MAL-V06-CORE-SPEC-010
- MAL-V03-B03-01-013
- MAL-V05-STD-001
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V05-STD-015

Version History

Version	Date	Status	Notes
v25 draft	2026-06-29	Draft	Initial executable core specification.